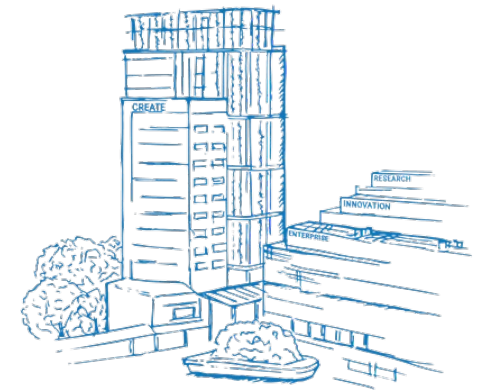


Agentic AI Workshop

Week 2 — From LLMs to Agents: why language models work, and the ladder that turns one into an agent.

Dr. Kerem Delikoyun · TUMCREATE · 3 June 2026



BLOCK 0 · 5 MIN

Where we left off

Last week: how we got here, plus Claude Code and Organon — working agents, built as plain files on disk. Today we open the hood.

Week 1 recap, in one slide

What Week 1 covered

- **Foundations** — two long paths (a 200-yr abstraction staircase + an 80-yr AI road) converge at natural language
- **Claude Code** — the CLI-native coding agent + its primitives: CLAUDE.md, tools, skills, sub-agents, MCP, hooks
- **Organon** — an agentic OS for scientists; memory, identity & skills as files on disk
- The equation: **agent = LLM + primitives** (memory, identity, tools, learnings)

Where Week 2 goes

1. *Open the box* — why LLMs work at all
2. *Climb the ladder* — bare model -> tools -> reasoning
3. *Pick up a framework* — LangChain
4. *Give it memory* — RAG

Five Colab notebooks back every idea on the right. You'll run them as we go.

PART 1 · WHY LLMS WORK

Opening the box

Six slides on what a language model actually is — so "agent" isn't magic stacked on magic.

The unlock: one architecture, three exponentials

For 60 years, language AI was hand-built rules and narrow models. Three things changed at once:

- **Architecture** — the Transformer (2017): attention replaces recurrence, so training parallelises across a whole sequence
- **Scale** — parameters from millions -> hundreds of billions
- **Data + compute** — train on much of the public web

THE BITTER LESSON

General methods that scale with compute beat clever hand-crafted ones. The Transformer was the first language architecture that kept getting better the more data and compute you fed it — no plateau in sight.

"Attention Is All You Need", Vaswani et al. 2017 — one of the five papers in today's RAG corpus.

Under the hood: the Transformer (2017)

Every modern LLM is built from this. Two stacks of identical blocks; each block is just three ideas:

- **Multi-Head Attention** — every token weighs what's relevant in all the others (the engine from the last slide)
- **Feed-Forward** — a small per-token network on top
- **Add & Norm** — residuals + normalisation that keep deep stacks trainable

Stack it **N**×, add embeddings + positional encoding — that's the model. Chat LLMs keep the right (decoder) stack.

WHY IT MATTERED

No recurrence: the whole sequence is processed in **parallel** — what made web-scale training possible, and the **T** in GPT.

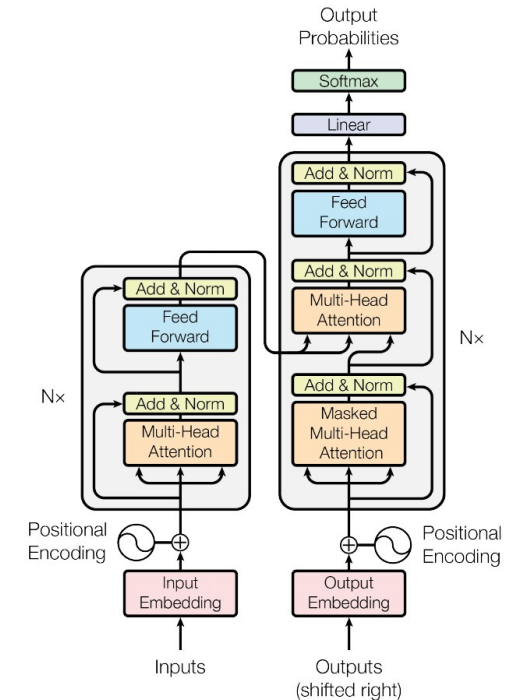


Figure 1: The Transformer - model architecture.

Figure 1, "Attention Is All You Need", Vaswani et al. 2017 — paper #2 in today's RAG corpus.

One dumb objective: predict the next token



Train on trillions of tokens with a single self-supervised target: **guess the next token**. No labels, no human annotation — the text is its own answer key.

Why this is enough.

To predict the next token well across *all* of human text, the model is forced to learn grammar, facts, translation, arithmetic, code, and a little reasoning.

COMPRESSION $\approx$ UNDERSTANDING

Predicting well means modelling the structure that *generated* the text. Squeezing the web into weights forces useful abstractions to fall out.

How text becomes math: tokens -> embeddings

Tokenisation — text is split into sub-word *tokens*.

```
"agentic" -> ["agent", "ic"]  
"RAG"     -> ["R", "AG"]
```

Embedding — each token maps to a vector in a high-dimensional space (hundreds–thousands of dims).

Meaning becomes *geometry*: tokens with similar meaning sit close together.

```
king - man + woman ~ queen  
Paris - France + Japan ~ Tokyo
```

WHY IT MATTERS TODAY

The **same** embedding idea powers RAG retrieval in Notebook 03 — we embed text chunks and find the nearest ones to a question.

The engine: attention as learned relevance

The scientist read the paper because **it** was ____

what “it” attends to — by learned relevance:



Each token builds its meaning by **attending** to the other tokens that matter — weighting them by learned relevance. Stack this dozens of times and you get rich, context-aware representations.

Two properties that changed everything:

- **Parallel** — the whole sequence is processed at once (RNNs went word-by-word). This is what made web-scale training feasible.
- **Long-range** — a token can attend to anything in the context window, not just its neighbours.

A Transformer block = attention + a small feed-forward network, repeated N times.

Scaling laws & emergence

Performance improves *predictably* with model size, data, and compute — smooth power-law curves (Kaplan 2020; Hoffmann/**Chinchilla** 2022).

CHINCHILLA, COMPUTE-OPTIMAL

For a fixed compute budget, scale parameters and data *together* — about **20 training tokens per parameter**. Most early models were badly under-trained.

Chinchilla is paper #3 in today's RAG corpus — you'll retrieve this exact claim in Notebook 03.

Emergence — some abilities are absent in small models and appear, fairly abruptly, past a scale threshold:

- in-context learning (learn from examples in the prompt, no weight update)
- multi-step arithmetic
- following instructions
- **chain-of-thought reasoning**

The capabilities we build on this week *emerged* from scale — nobody hand-coded them.

Why this architecture fits us so well

Language is humanity's API. We already encode knowledge, intent, and reasoning in text. A model fluent in text plugs straight into how we work — no new interface to learn.

In-context learning $\approx$ working memory. Put examples or facts in the prompt and behaviour adapts on the spot, like a person holding instructions in mind for one task.

SYSTEM 1 VS SYSTEM 2 (KAHNEMAN)

System 1 — fast, automatic, intuitive. A raw next-token answer is System-1-like: one fluent pass, no deliberation.

System 2 — slow, effortful, step-by-step. We *elicit* it by asking the model to reason out loud.

That single idea — **make the fast model slow down and deliberate** — is the seed of every reasoning pattern in Part 2.

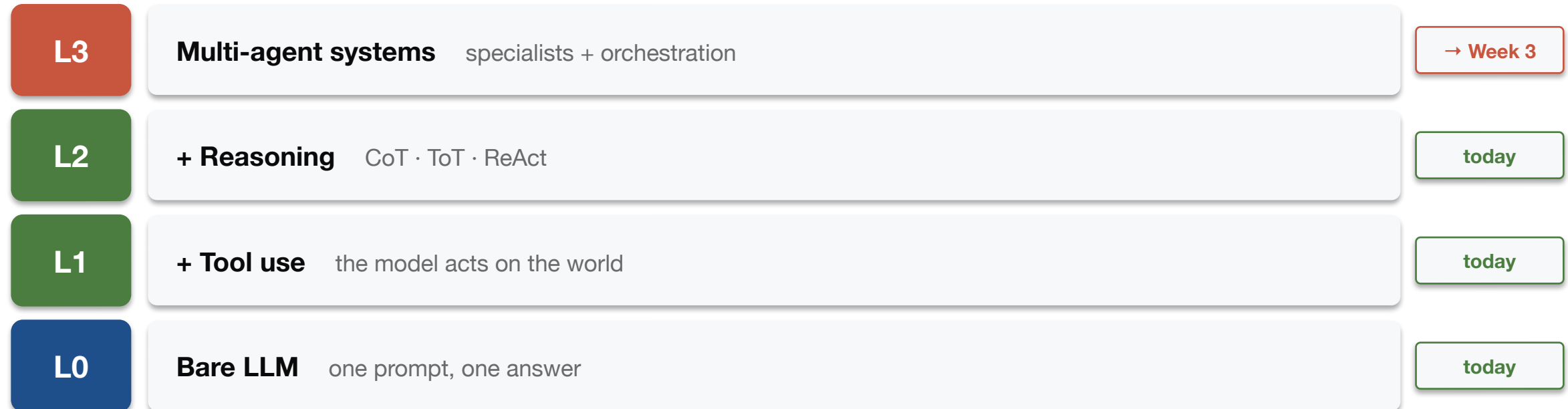
Analogy, not a claim that models literally think. It's a useful map for *why* the patterns work.

PART 2 · SINGLE AGENTS

The agentic ladder

From a bare model to a reasoning, tool-using agent — one rung at a time. All single-agent this week; multi-agent is Week 3.

The ladder — what we climb today

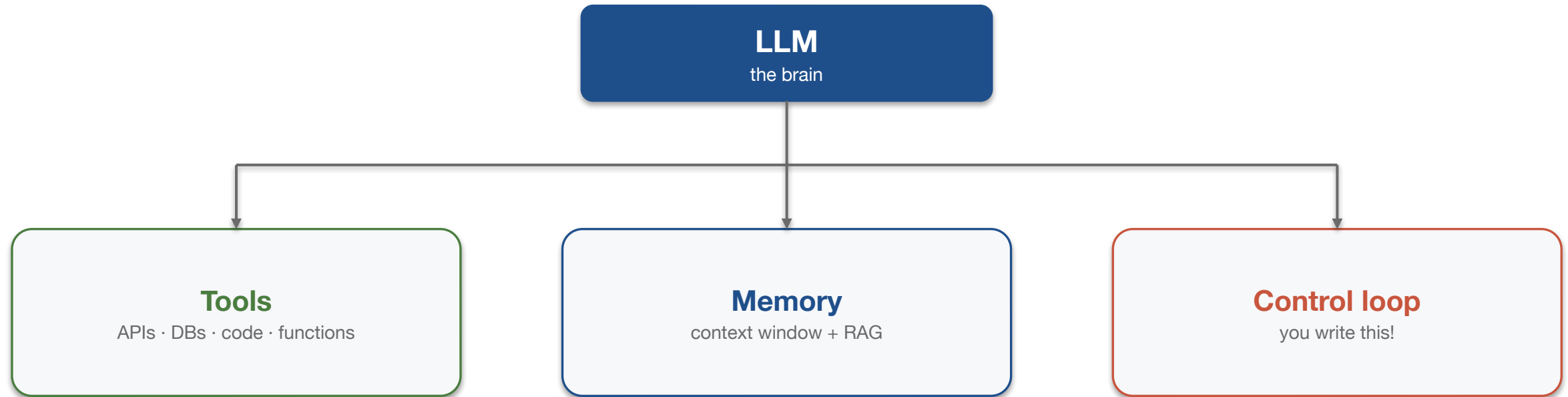


Each rung **adds one capability** to the rung below.
Nothing here is a new model — it's the same LLM with more scaffolding around it.

THE THREAD

L0 answers. L1 lets it *act*. L2 lets it *deliberate*. The "agent" is the loop you wrap around the model — not the model itself.

Anatomy of an agent



Tools = capability. **Memory** = state. **Loop** = agency. Take away the loop and you have an LLM call. Take away tools and you have a chatbot. Take away memory and you have amnesia.

L0 — the bare LLM

One prompt in, one answer out. Brilliant at language; blind beyond its training.

What it cannot do:

- Know anything after its training cut-off
- Look up a fact it didn't memorise
- Do anything in the world (send mail, query a DB, run code)
- Reliably do long exact arithmetic
- Tell you *why* it's confident

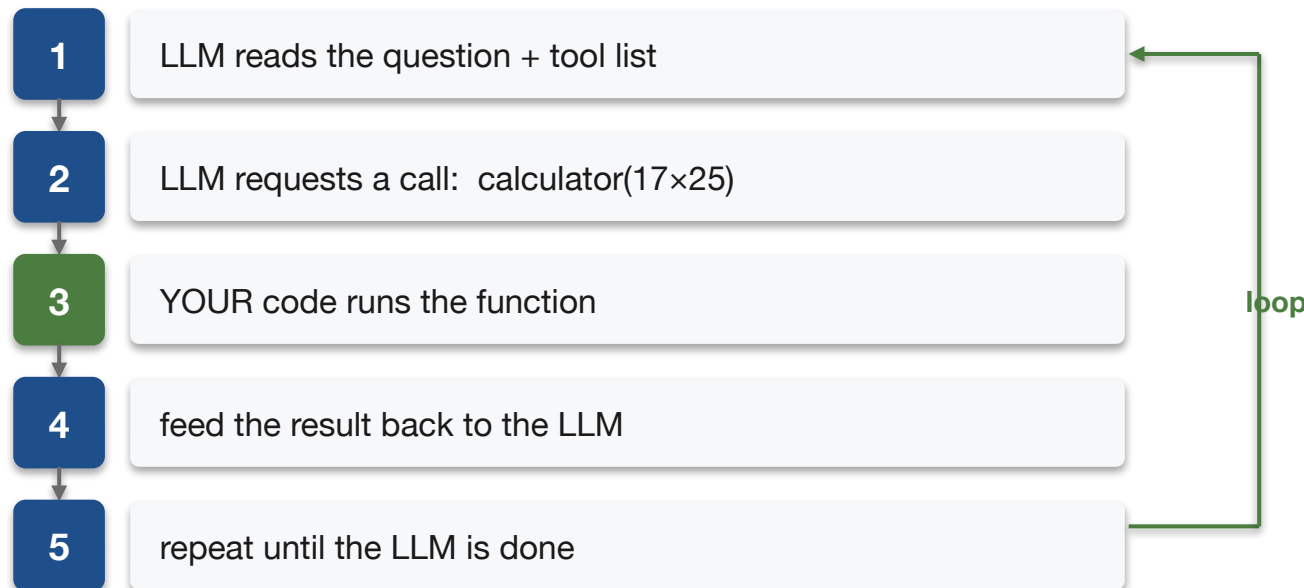
user -> LLM -> answer

THE CEILING

A bare LLM is a brilliant *improviser* with no hands, no memory of today, and no scratchpad. Everything above this rung removes one of those limits.

L1 — tool use: give the model hands

A **tool** is just a function the model may ask you to call. The loop:



Function calling. Modern models are fine-tuned to pick a tool and fill its arguments from a schema you provide.

WHO CARRIES THE RESPONSIBILITY?

The **vendor** trained the tool-selection skill. **You** own writing clear tool names + descriptions. Get those right and the model routes well.

Suddenly the model can fetch fresh facts, query data, and act — L0's ceiling is gone.

The model never runs anything. It *requests*; your code executes and returns the result.

L2 — reasoning: make the fast model slow down

The jump from L1 to L2 is **System 1** -> **System 2**. Same model, but we change the prompt so it *deliberates* before it answers or acts.

CHAIN-OF-THOUGHT

Think in a straight line, out loud.

TREE-OF-THOUGHTS

Branch, evaluate, search — don't bet on the first idea.

REACT

Reason *and* act — interleave thinking with tool calls.

CoT is the base. ToT searches over it. ReAct couples it to L1's tools. The next four slides take each in turn.

Chain-of-Thought (CoT)

Five words change the answer: "**Let's think step by step.**"

Q: A juggler has 16 balls. Half are golf balls,
and half of those are blue. How many blue
golf balls?

DIRECT: 4 (right or wrong, no trace)

STEP BY STEP:
16 balls -> half golf = 8 golf balls
8 golf -> half blue = 4 blue golf balls
Answer: 4 (visible, checkable)

The model uses its own intermediate tokens as a **scratchpad** — deliberate reasoning, summoned by a prompt.

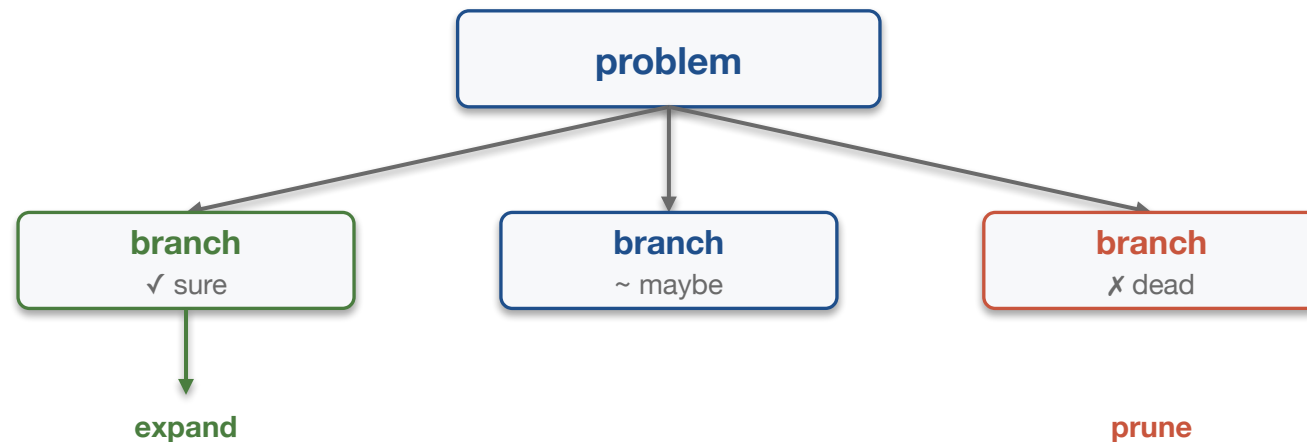
WHY IT WORKS

Hard problems need intermediate steps. Forcing the model to write them spends compute *before* the answer, instead of guessing in one pass.

CoT is System 2, on demand. It's the building block ReAct extends.

Tree-of-Thoughts (ToT)

CoT commits to one line of reasoning. If an early step is wrong, the whole chain is wrong. **ToT treats reasoning as search.**



Propose -> evaluate -> expand the best, with backtracking out of dead ends.

THE TRADE

ToT spends a lot more compute (many model calls) to buy accuracy on problems where one CoT chain routinely fails — planning, puzzles, proof search.

Notebook 02 shows one propose/evaluate level live on the Game of 24.

ReAct = Reason + Act

CoT thinks. L1 acts. **ReAct interleaves them** in one loop (Yao et al. 2022): a Thought, an Action (tool call), the Observation it returns — repeat.

```
Answer the question. Tools: {tools}

Question: the input question
Thought: reasoning about what to do next
Action: one of [{tool_names}]
Action Input: the input to the action
Observation: the result of the action
... (repeat) ...
Thought: I now know the final answer
Final Answer: ...
```

There is no "ReAct API". It's English in a fixed format that the model produces and we parse.

THE MECHANISM

Stop the model right after "Action Input". Run the tool yourself. Append the real "Observation". Re-prompt. The model never hallucinates a result — it reads the one your code returned.

What a ReAct trace looks like

```
Thought: I need the authors of the ReAct paper first.  
Action: web_search  
Action Input: ReAct paper LLM reasoning acting authors  
Observation: Yao et al., ICLR 2023 (arXiv:2210.03629).  
  
Thought: Now the year of the transformer paper.  
Action: web_search  
Action Input: transformer paper Attention Is All You Need year  
Observation: Published in 2017 (Vaswani et al.).  
  
Thought: I have both. 2023 – 2017.  
Action: calculator  
Action Input: 2023 – 2017  
Observation: 6  
  
Thought: I now know the final answer.  
Final Answer: ReAct (Yao et al., 2023) came six years after  
the transformer paper (Vaswani et al., 2017).
```

Thought -> Action -> Observation, repeat. That loop *is* the agent. You'll build it by hand in Notebook 02.

Two ways to pick a tool

ReAct agent

Tool choice lives in the **prompt** you write. The model emits `Action: / Action Input:` as text; you parse it with a regex.

- **High control** — read and edit every step
- Brittle parsing; works even on models with no native tool API

Function-calling agent

Tool choice is the **model vendor's** native capability. It returns a structured tool call; no parsing.

- **Less control**, much less headache
- Robust; needs a modern tool-calling model

"WHERE DO WE SHIFT THE RESPONSIBILITY?"

ReAct keeps tool selection in *your* prompt; the function-calling agent shifts it to the *vendor*. Most production agents use function calling; ReAct remains the clearest way to *understand* what an agent is.

PART 3 · THE FRAMEWORK

A short intro to LangChain

We've climbed the ladder by hand. LangChain is the standard toolkit that implements every rung — so you don't re-write the loop each time.

What LangChain is (and isn't)

A thin, **vendor-neutral** layer over "call a model, give it a prompt, maybe tools, parse the output." The same code runs on Anthropic, OpenAI, Google.

Five pieces, one operator:

Piece	Class
Model wrapper	ChatAnthropic
Messages	SystemMessage / HumanMessage
Prompt template	ChatPromptTemplate
Output parser	StrOutputParser
Tool	@tool

LCEL — the pipe. Connect components with `|`, like a Unix pipe:

```
chain = prompt | llm | parser
chain.invoke({"concept": "embeddings"})
```

IT ISN'T MAGIC

Everything we built by hand — the tool loop, the ReAct parse — LangChain wraps in one call. Because you built the primitive first, the abstraction is transparent.

LangChain implements the ladder

Rung	By hand (Parts 1-2)	In LangChain
L0 bare model	<code>client.messages.create(...)</code>	<code>ChatAnthropic(...).invoke(...)</code>
L1 tool use	your for loop + regex	<code>create_tool_calling_agent + AgentExecutor</code>
L2 ReAct	your prompt + <code>parse_action</code>	<code>create_react_agent + AgentExecutor</code>
Memory (RAG)	manual chunk + embed + search	loaders + splitters + vector stores + retrievers

We pin the classic LangChain 0.3.x line — the last with `AgentExecutor`. `LangGraph` (the newer agent runtime) is the Week 3 reveal.

Structured output: function calling, aimed at the answer

```
from pydantic import BaseModel, Field
from typing import Literal

class ClaimCheck(BaseModel):
    verdict: Literal["true", "false", "uncertain"]
    confidence: Literal["low", "medium", "high"]
    reason: str = Field(description="<= 15 words")

checker = llm.with_structured_output(ClaimCheck)
result = checker.invoke("The transformer paper is from 2017. True?")

result.verdict      # -> "true" (a typed object, not a string)
```

Point a schema at the **answer** instead of a tool, and the model fills it in. You get back a **validated Python object** — no regex, no "please respond in JSON".

SAME MACHINERY

It's the exact function-calling mechanism from the tool-calling agent. Reach for it whenever an agent's output feeds the next pipeline step and must have predictable fields.

Run live in Notebook 01.

From demo to production

REAL TOOLS — TAVILY

Our notebooks mock web_search so they run offline. In production, drop in a hosted search built for agents — same tool interface, real results:

```
search = TavilySearchResults(max_results=3)
agent = create_react_agent(llm, [search], prompt)
```

```
langchain_community.tools.tavily_search
```

OBSERVABILITY — LANGSMITH

Set two env vars and every agent run is traced step-by-step — each Thought, Action, tool latency, token count — with no code change:

```
LANGCHAIN_TRACING_V2 = "true"
LANGCHAIN_API_KEY    = "ls-..."
```

You debug an agent by **watching the trace**, not guessing.

Hands-on: three notebooks, back to back

NOTEBOOK 00 · LANGCHAIN IN 15 MIN

Model wrapper, messages, prompt templates, the LCEL pipe, first @tool.

NOTEBOOK 01 · TOOL USE

The smallest agent, three ways: hand-rolled -> native tool-use -> LangChain tool-calling agent.

NOTEBOOK 02 · REACT, CoT, ToT

CoT -> hand-built ReAct loop -> `create_react_agent` -> ToT -> the function-calling-vs-ReAct contrast.

Same model. Same loop you just saw on slides. Now you watch it run, step by step.

► [Switch to Colab — Notebooks 00, 01, 02](#)

PART 4 · MEMORY

RAG end-to-end

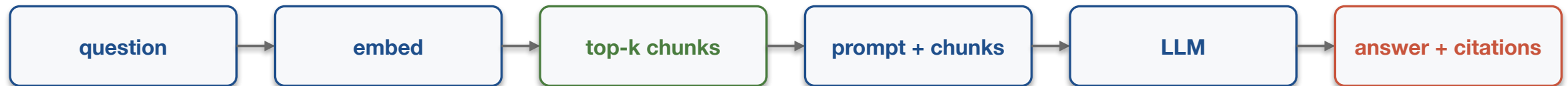
Retrieval-Augmented Generation gives the agent memory it can cite. Two parts — both visible — compared three ways.

RAG in one diagram

INDEX (once)



QUERY (every question)



Two halves. **Retrieval** finds the relevant chunks (same embedding idea from Part 1). **Augmentation** stuffs them into the prompt. The model's job becomes "answer from this context, and cite it."

Build it with LangChain pieces

Each pipeline stage is one LangChain class:

Stage	Class
Load PDFs	PyPDFLoader
Split	RecursiveCharacterTextSplitter
Embed	HuggingFaceEmbeddings
Store (dense)	FAISS · Chroma
Store (sparse)	BM25Retriever
Fuse	EnsembleRetriever

FAISS vs Chroma — both local, both `.as_retriever()`:

FAISS

Fastest pure-vector index, lives in RAM. Great default.

CHROMA

A small persistent local DB — write once, reopen later, no re-embedding.

Same retriever interface — the rest of the pipeline doesn't care which you pick.

Three retrieval strategies

Dense (FAISS / Chroma)

- Embed query + chunks, cosine similarity
- Great for paraphrase & synonyms
- "What does *X mean*?"

Sparse (BM25)

- Classical keyword scoring, no embeddings
- Great for exact terms
- "What about *RLAIF*?"

Hybrid (EnsembleRetriever)

- Fuses both with Reciprocal Rank Fusion
- Catches paraphrase **and** exact-term
- Usually wins on section-level hit

NOTEBOOK 04 APPENDIX

The same recipe on real Elasticsearch with native RRF — production substrate, same idea.

The dirty secret of RAG: chunking dominates

Fixed-window — split every N chars

- Fast; breaks sentences mid-thought

Recursive — paragraph -> sentence -> word

- Respects boundaries; LangChain default
- Better for dense *and* sparse

Semantic — split on embedding-distance jumps

- Slowest, sometimes best for narrative; we mention, don't build

NOTEBOOK 03 · CHUNK + OVERLAP

137 pages -> **676 chunks** at 800 chars / 100 overlap.

Chunk size and overlap usually move retrieval quality *more* than the choice of embedding model. Sweep them on your own corpus.

For the workshop we use **recursive** — the cheapest win.

Comparison: hit@5 on five questions

Test set — 5 questions, each labelled with its ground-truth paper + section keywords. Two metrics:

- **paper-hit@5** — right *paper* in the top 5?
- **section-hit@5** — right *part* of it?

The first is easy; the second is honest.

Notebook 03 run (5-paper corpus):

	paper-hit	section-hit
Dense	5/5	5/5
BM25	5/5	5/5
Hybrid	5/5	5/5

On a small, clean corpus all three nail it. On a big messy one, hybrid pulls ahead — it catches both query types.

RAG vs raw LLM — same question

RAW LLM

"The ReAct loop interleaves reasoning and action..."

No citation. Possibly wrong on specifics. Sounds confident either way.

RAG

"...interleaving reasoning, action and observation improves groundedness **[react_yao_2022.pdf#540]**."

Every claim points to a chunk you can re-read.

Citations are the killer feature — not prompt-engineering magic, just retrieval plus telling the model to cite its source.

► Switch to Colab — Notebook 03 (run the comparison, then ask a grounded question)

PART 5 • 10 MIN

What's next — Week 3

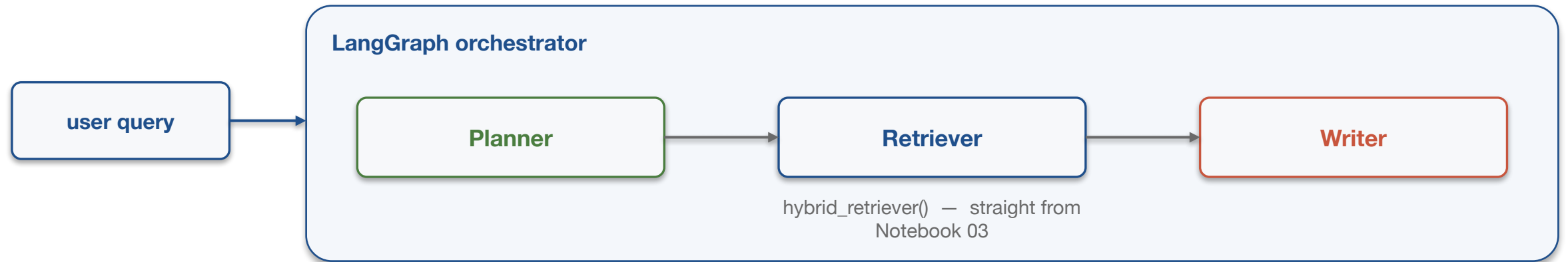
A single ReAct + RAG agent covers a lot. Specialisation is the honest reason to reach for more than one.

Limits of the single-agent pattern

Limit	Symptom	Fix in W3
Context overflow	Long reasoning traces exhaust the window	Hand subtasks to specialists
No specialisation	One prompt does everything, none of it well	Planner / Retriever / Writer split
Sequential bottleneck	Parallel-decomposable work runs serially	Graph orchestration
Hard to evaluate	Can't isolate which step failed	Per-node metrics

These are the **L3** rung — multi-agent systems — that we deliberately left off the ladder today.

Today's retriever becomes a W3 tool node



The retrieval *function* doesn't change — the orchestrator around it does. Same with your ReAct loop: it becomes a node in a graph.

What to read before Week 3

- **LangGraph quickstart** — `langchain-ai/langgraph` repo
- **Anthropic's "Building effective agents"** — the workflows-vs-agents section
- **GPT-Researcher** — a production multi-agent deep-research pattern
- Your **Notebook 02 ReAct trace** — bring it back; we'll convert it to a graph

WRAP-UP

Take it home

Five notebooks, one mental model: the loop is the agent.

What you take home

- **Notebook 00** — LangChain in 15 minutes
- **Notebook 01** — tool use, three ways (hand-rolled -> native -> tool-calling agent)
- **Notebook 02** — CoT, ReAct from scratch & via `create_react_agent`, ToT, FC-vs-ReAct
- **Notebook 03** — full RAG with FAISS + Chroma + hybrid retrieval
- **Notebook 04** — optional Elasticsearch appendix

All open in Colab from github.com/krmde1/agentic-ai-for-scientists-workshop/week-02-patterns

One sentence to remember

THE WEEK 2 TAKEAWAY

A language model predicts the next token. An **agent** is the loop you wrap around it — tools give it hands, reasoning gives it a scratchpad, RAG gives it memory it can cite. Frameworks add ergonomics; they don't add the agency.

Questions?

Kerem Delikoyun · TUMCREATE

kerem.delikoyun@tum-create.edu.sg · github.com/krmDEL/Agentic-AI-for-Scientists-Workshop